Hair Fix Salon Technical Design Document

Online Appointment Booking Platform

Item	Information
Prepared by	Robert Martinez-Olivera and Yaswanth Gosukonda
Course and section	DATABASE SYS IMPLEMT 01:198:437:01
Date	October 20, 2026
TDD version	1.0
Source requirements	Hair Fix Salon FRD version 2.1
Status	Draft for team and instructor review

Document Purpose

This document converts the approved Hair Fix Salon functional requirements into an implementation-ready MySQL design. It defines the database platform, architecture, ERD, 14-table relational schema, column data types, keys, constraints, controlled procedures, reports, indexes, security assumptions, tests, and requirement traceability. Another student should be able to build and test the database from this blueprint without guessing the major design decisions.

Design conclusion The database will use a normalized single-schema design with controlled transaction procedures for booking, rescheduling, cancellation, status changes, schedules, and payments. Appointment history is append-only, and booked price and duration are intentional historical snapshots.

1 Project Identification

Item	Project information
Project title	Hair Fix Salon Online Appointment Booking Platform
Prepared by	Robert Martinez-Olivera and Yaswanth Gosukonda
Course and section	DATABASE SYS IMPLEMT 01:198:437:01
Date	October 20, 2026
Version	1.0

2 Design Overview

This project implements a relational database for Hair Fix Salon. The database manages roles, accounts, customer and staff profiles, one seeded location, services, staff qualifications, work schedules, appointments, immutable appointment history, status transitions, payment methods, and payment transactions. It uses MySQL 8.x and supports availability lookup, controlled booking and rescheduling, cancellation, staff status updates, non-sensitive payment recording, and seven operational reports.

The design contains 14 related tables. Primary keys uniquely identify rows; foreign keys preserve valid relationships; unique constraints prevent duplicate emails, services, assignments, transition definitions, and external payment references; check constraints validate domain and range values; and stored procedures enforce rules that depend on multiple tables, authorization, time, or transaction locking.

3 Technical Environment

Component	Selected technology or tool	Purpose
Database management system	MySQL 8.x	Store relational data and execute SQL
Database client and IDE	MySQL Workbench 8.x	Create schema, run scripts, inspect data, and test queries
Modeling tool	MySQL Workbench EER or draw.io	Maintain the implementation ERD
SQL language	MySQL SQL	Define tables, constraints, objects, data, and reports
Documentation	Microsoft Word or Google Docs	Maintain the FRD and TDD
Source control	GitHub if used by the team	Version SQL scripts and documentation
Operating environment	Windows, macOS, or Linux	Local development and testing
Test data	Fictional sample records	Demonstrate functions without sensitive personal data

4 Architecture and Scope

The solution uses a database-centered relational architecture. MySQL stores normalized operational records, enforces referential integrity, runs controlled business transactions, and returns reporting results. A future web, mobile, or desktop interface may call the stored procedures and views, but user-interface development is outside this project.

- One MySQL schema named hair_fix_salon contains all project objects.
- Version 1.0 uses one seeded Hair Fix Salon location and the America/New_York time zone.
- Each appointment has one customer, staff member, service, location, and current status.
- The database records password hashes and non-sensitive payment history but not plaintext passwords or complete card credentials.

- A production website, real payment processing, notifications, payroll, inventory, retail sales, multi-location workflows, and enterprise backup automation are outside scope.
- All demonstration data is fictional.

5 Entity Relationship Design

The ERD separates account identity from role-specific profiles, resolves staff-to-service qualifications with a junction table, stores allowed status changes as data, and preserves one-to-many audit and payment histories for each appointment.

Entity	Purpose	Primary key	Important relationships
Role	Seeded primary roles	RoleID	One Role to many UserAccount rows
UserAccount	Identity, login, name, role, and status	UserID	Optional one-to-one Customer or Staff profile
Customer	Customer-specific profile	CustomerID	One customer to many appointments
Location	Seeded salon reference data	LocationID	One location to many staff and appointments
Staff	Staff-specific profile	StaffID	Schedules, qualifications, and appointments
Service	Service catalog and current values	ServiceID	Qualifications and appointments
StaffService	Staff-service qualification junction	StaffID and ServiceID	Links Staff and Service
StaffSchedule	Availability and blocked intervals	ScheduleID	Many schedule rows per staff member
AppointmentStatus	Seeded status vocabulary	StatusID	Current and historical status references
AppointmentStatusTransition	Allowed next statuses by actor type	FromStatusID and ToStatusID	Self-referencing status junction
Appointment	Current booking state and snapshots	AppointmentID	Customer, staff, service, location, status
AppointmentHistory	Append-only audit events	HistoryID	Many events per appointment and actor
PaymentMethod	Allowed non-sensitive payment methods	PaymentMethodID	One method to many transactions
PaymentTransaction	Charge and refund history	TransactionID	Many transactions per appointment

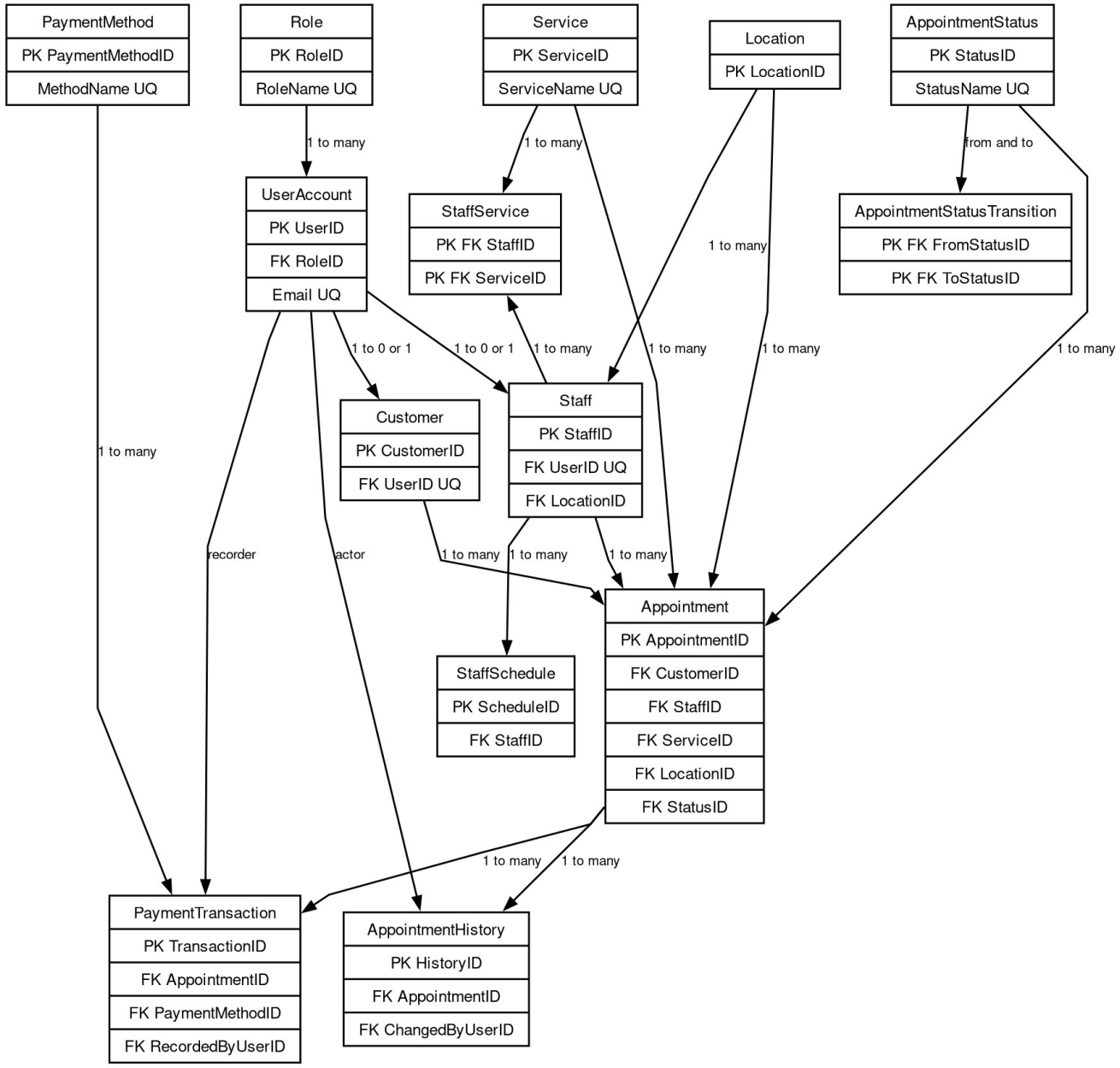


Figure 1 Hair Fix Salon implementation ERD

Relationship Narrative

- One Role may be assigned to zero, one, or many UserAccount rows; each account has exactly one role.
- One UserAccount may have zero or one Customer profile and zero or one Staff profile. Role validation prevents an account from receiving the wrong profile type.
- One Location may be referenced by many Staff and Appointment rows; each staff member and appointment references exactly one location.
- Staff and Service have a many-to-many relationship resolved by StaffService.
- One Staff member may have many StaffSchedule rows and many assigned appointments.
- One Customer may own many appointments; each appointment belongs to exactly one customer.
- Each appointment references exactly one service and one current AppointmentStatus.
- AppointmentStatusTransition resolves allowed directed transitions between two AppointmentStatus rows.
- One Appointment may have many AppointmentHistory rows and many PaymentTransaction rows.
- Each history event identifies one actor in UserAccount, and each payment transaction identifies one recorder in UserAccount and one PaymentMethod.

6 Relational Schema

```
Role(RoleID PK, RoleName UQ, Description)

UserAccount(UserID PK, Email UQ, PasswordHash, FirstName, LastName, RoleID FK -> Role, AccountStatus, CreatedAt, LastUpdatedAt)

Customer(CustomerID PK, UserID UQ FK -> UserAccount, Phone, ContactPreference, ProfileStatus)

Location(LocationID PK, SalonName, AddressLine1, AddressLine2, City, StateCode, PostalCode, Phone, TimeZone, ActiveFlag)

Staff(StaffID PK, UserID UQ FK -> UserAccount, Phone, Specialty, ActiveFlag, HireDate, LocationID FK -> Location)

Service(ServiceID PK, ServiceName UQ, Description, CurrentPrice, DurationMinutes, ActiveFlag)

StaffService(StaffID PK FK -> Staff, ServiceID PK FK -> Service, IsActive, AssignedAt)

StaffSchedule(ScheduleID PK, StaffID FK -> Staff, ScheduleType, DayOfWeek, ScheduleDate, StartTime, EndTime, EffectiveStartDate, EffectiveEndDate, Reason, CreatedAt)

AppointmentStatus(StatusID PK, StatusName UQ, IsFinal, SortOrder UQ)

AppointmentStatusTransition(FromStatusID PK FK -> AppointmentStatus, ToStatusID PK FK -> AppointmentStatus, CustomerAllowed, StaffAllowed, AdministratorAllowed, RequiresReason)

Appointment(AppointmentID PK, CustomerID FK -> Customer, StaffID FK -> Staff, ServiceID FK -> Service, LocationID FK -> Location, StartDateTime, EndDateTime, BookedPrice, BookedDurationMinutes, StatusID FK -> AppointmentStatus, CreatedAt, UpdatedAt, CancelledAt, Notes)

AppointmentHistory(HistoryID PK, AppointmentID FK -> Appointment, EventType, ChangedByUserID FK -> UserAccount, ChangedAt, PreviousStatusID FK, NewStatusID FK, previous and new time values, previous and new StaffID FK, previous and new ServiceID FK, Reason, RequestID)

PaymentMethod(PaymentMethodID PK, MethodName UQ, ActiveFlag)

PaymentTransaction(TransactionID PK, AppointmentID FK -> Appointment, Amount, TransactionType, PaymentMethodID FK -> PaymentMethod, PaymentStatus, TransactionDate, RecordedByUserID FK -> UserAccount, CreatedAt, ExternalReference UQ)
```

7 Table and Attribute Specifications

The following data dictionary defines every table and column planned for implementation.

Table Role

Purpose Stores the three seeded application roles. One row represents one primary role.

Column	Data type	Null allowed	Key or constraint	Description
RoleID	TINYINT UNSIGNED	No	PK, AUTO_INCREMENT	Role identifier
RoleName	VARCHAR(30)	No	UNIQUE	Customer, Staff, or Administrator
Description	VARCHAR(255)	Yes	None	Human-readable role purpose

Rules implemented Implements BR-02 and supports FR-01 and FR-02. Seed values are controlled reference data.

Table UserAccount

Purpose Stores identity, login, name, role, and account status. One row represents one registered account.

Column	Data type	Null allowed	Key or constraint	Description
UserID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Account identifier
Email	VARCHAR(254)	No	UNIQUE, CI collation	Login email; case-insensitive uniqueness
PasswordHash	VARCHAR(255)	No	NOT NULL	One-way password hash only
FirstName	VARCHAR(60)	No	NOT NULL	Account holder first name
LastName	VARCHAR(60)	No	NOT NULL	Account holder last name
RoleID	TINYINT UNSIGNED	No	FK -> Role	One primary role

Column	Data type	Null allowed	Key or constraint	Description
AccountStatus	VARCHAR(12)	No	CHECK	Active, Inactive, or Locked
CreatedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Creation timestamp
LastUpdatedAt	DATETIME	No	ON UPDATE	Most recent account update

Rules implemented Implements FR-01, FR-02, BR-01, BR-02, and BR-03. Names are stored here once and are not repeated in Customer or Staff.

Table Customer

Purpose Stores customer-specific profile data. One row represents one account with the Customer role.

Column	Data type	Null allowed	Key or constraint	Description
CustomerID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Customer identifier
UserID	BIGINT UNSIGNED	No	FK -> UserAccount, UNIQUE	One-to-one account link
Phone	VARCHAR(25)	No	NOT NULL	Required customer phone
ContactPreference	VARCHAR(10)	No	CHECK	Email or Phone
ProfileStatus	VARCHAR(12)	No	CHECK	Active or Inactive

Rules implemented Implements FR-03 and BR-25. A controlled insert verifies that the linked account has the Customer role.

Table Location

Purpose Stores salon reference data. Version 1.0 uses one seeded Hair Fix Salon row.

Column	Data type	Null allowed	Key or constraint	Description
LocationID	SMALLINT UNSIGNED	No	PK, AUTO_INCREMENT	Location identifier
SalonName	VARCHAR(100)	No	NOT NULL	Hair Fix Salon
AddressLine1	VARCHAR(120)	No	NOT NULL	Street address
AddressLine2	VARCHAR(120)	Yes	None	Suite or unit
City	VARCHAR(60)	No	NOT NULL	City
StateCode	CHAR(2)	No	NOT NULL	Two-letter state code
PostalCode	VARCHAR(10)	No	NOT NULL	Postal code
Phone	VARCHAR(25)	No	NOT NULL	Salon phone
TimeZone	VARCHAR(64)	No	NOT NULL	America/New_York
ActiveFlag	BOOLEAN	No	DEFAULT TRUE	Controls use in new bookings

Rules implemented Implements BR-20, BR-22, and the single-location scope decision. Deactivation does not delete historical appointments.

Table Staff

Purpose Stores staff-specific profile data. One row represents one account with the Staff role.

Column	Data type	Null allowed	Key or constraint	Description
StaffID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Staff identifier
UserID	BIGINT UNSIGNED	No	FK -> UserAccount, UNIQUE	One-to-one account link

Column	Data type	Null allowed	Key or constraint	Description
Phone	VARCHAR(25)	No	NOT NULL	Staff contact phone
Specialty	VARCHAR(100)	Yes	None	Optional specialty description
ActiveFlag	BOOLEAN	No	DEFAULT TRUE	Eligibility for new bookings
HireDate	DATE	No	NOT NULL	Employment start date
LocationID	SMALLINT UNSIGNED	No	FK -> Location	Seeded salon location

Rules implemented Implements FR-04 and BR-20. A controlled insert verifies that the linked account has the Staff role.

Table Service

Purpose Stores the salon service catalog and current booking values.

Column	Data type	Null allowed	Key or constraint	Description
ServiceID	INT UNSIGNED	No	PK, AUTO_INCREMENT	Service identifier
ServiceName	VARCHAR(100)	No	UNIQUE	Unique active catalog name
Description	VARCHAR(500)	Yes	None	Customer-facing description
CurrentPrice	DECIMAL(10,2)	No	CHECK >= 0	Current price for new bookings
DurationMinutes	SMALLINT UNSIGNED	No	CHECK > 0	Current duration for new bookings
ActiveFlag	BOOLEAN	No	DEFAULT TRUE	Eligibility for new bookings

Rules implemented Implements FR-05, BR-04, and BR-20. Price and duration changes do not alter snapshots in existing appointments.

Table StaffService

Purpose Resolves the many-to-many qualification relationship between Staff and Service.

Column	Data type	Null allowed	Key or constraint	Description
StaffID	BIGINT UNSIGNED	No	PK, FK -> Staff	Qualified staff member
ServiceID	INT UNSIGNED	No	PK, FK -> Service	Qualified service
IsActive	BOOLEAN	No	DEFAULT TRUE	Whether qualification may be used
AssignedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Qualification assignment time

Rules implemented Implements FR-04 and BR-05. The composite primary key prevents duplicate staff-service assignments.

Table StaffSchedule

Purpose Stores recurring availability, date-specific availability, and blocked time for a staff member.

Column	Data type	Null allowed	Key or constraint	Description
ScheduleID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Schedule record identifier
StaffID	BIGINT UNSIGNED	No	FK -> Staff	Staff member whose time is defined
ScheduleType	VARCHAR(30)	No	CHECK	RecurringAvailability, DateAvailability, or BlockedTime
DayOfWeek	TINYINT UNSIGNED	Yes	CHECK 1 to 7	Required for recurring availability

Column	Data type	Null allowed	Key or constraint	Description
ScheduleDate	DATE	Yes	Conditional CHECK	Required for date-specific or blocked time
StartTime	TIME	No	NOT NULL	Interval start
EndTime	TIME	No	CHECK EndTime > StartTime	Interval end
EffectiveStartDate	DATE	Yes	Conditional CHECK	Recurring effective start
EffectiveEndDate	DATE	Yes	CHECK	Optional recurring effective end
Reason	VARCHAR(255)	Yes	None	Reason for blocked or exceptional time
CreatedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

Rules implemented Implements FR-06, FR-22, BR-15, and BR-16. Availability logic applies BlockedTime before DateAvailability before RecurringAvailability.

Table AppointmentStatus

Purpose Stores the seeded appointment status vocabulary and final-state indicator.

Column	Data type	Null allowed	Key or constraint	Description
StatusID	TINYINT UNSIGNED	No	PK, AUTO_INCREMENT	Status identifier
StatusName	VARCHAR(30)	No	UNIQUE	Scheduled, Confirmed, In Progress, Completed, Cancelled, or No-Show
IsFinal	BOOLEAN	No	DEFAULT FALSE	True for Completed, Cancelled, and No-Show
SortOrder	TINYINT UNSIGNED	No	UNIQUE	Display order

Rules implemented Implements BR-11. Final states cannot return to active states through ordinary procedures.

Table AppointmentStatusTransition

Purpose Stores allowed next statuses, role permissions, and reason requirements for controlled status changes.

Column	Data type	Null allowed	Key or constraint	Description
FromStatusID	TINYINT UNSIGNED	No	PK, FK -> AppointmentStatus	Current status
ToStatusID	TINYINT UNSIGNED	No	PK, FK -> AppointmentStatus	Requested next status
CustomerAllowed	BOOLEAN	No	DEFAULT FALSE	Customer permission flag
StaffAllowed	BOOLEAN	No	DEFAULT FALSE	Assigned-staff permission flag
AdministratorAllowed	BOOLEAN	No	DEFAULT TRUE	Administrator permission flag
RequiresReason	BOOLEAN	No	DEFAULT FALSE	Reason required for this transition

Rules implemented Implements FR-11, FR-12, FR-19, FR-21, BR-12, and BR-14. The composite key prevents duplicate transition definitions.

Table Appointment

Purpose Stores the current state of one booked service with one customer, staff member, service, and location.

Column	Data type	Null allowed	Key or constraint	Description
AppointmentID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Appointment identifier
CustomerID	BIGINT UNSIGNED	No	FK -> Customer	Customer owner
StaffID	BIGINT UNSIGNED	No	FK -> Staff	Assigned staff member
ServiceID	INT UNSIGNED	No	FK -> Service	Single booked service
LocationID	SMALLINT UNSIGNED	No	FK -> Location	Seeded salon location
StartDateTime	DATETIME	No	NOT NULL	Appointment start in salon time
EndDateTime	DATETIME	No	CHECK	Start plus booked duration
BookedPrice	DECIMAL(10,2)	No	CHECK >= 0	Historical price snapshot
BookedDurationMinutes	SMALLINT UNSIGNED	No	CHECK > 0	Historical duration snapshot
StatusID	TINYINT UNSIGNED	No	FK -> AppointmentStatus	Current approved status
CreatedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Booking creation time
UpdatedAt	DATETIME	No	ON UPDATE	Most recent appointment change
CancelledAt	DATETIME	Yes	None	Cancellation timestamp when applicable
Notes	VARCHAR(500)	Yes	None	Non-sensitive appointment notes

Rules implemented Implements FR-07 through FR-12, FR-18 through FR-23, and BR-05 through BR-14. Overlap, qualification, activity, schedule, ownership, cutoff, and transition rules are enforced by controlled procedures.

Table AppointmentHistory

Purpose Stores append-only audit events for booking, rescheduling, cancellation, status change, and correction.

Column	Data type	Null allowed	Key or constraint	Description
HistoryID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	History event identifier
AppointmentID	BIGINT UNSIGNED	No	FK -> Appointment	Appointment being audited
EventType	VARCHAR(30)	No	CHECK	Booked, Rescheduled, Cancelled, StatusChanged, or Correction
ChangedByUserID	BIGINT UNSIGNED	No	FK -> UserAccount	Actor responsible for the event
ChangedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Event timestamp
PreviousStatusID	TINYINT UNSIGNED	Yes	FK -> AppointmentStatus	Status before event
NewStatusID	TINYINT UNSIGNED	Yes	FK -> AppointmentStatus	Status after event
PreviousStartDateTime	DATETIME	Yes	None	Start before event
NewStartDateTime	DATETIME	Yes	None	Start after event
PreviousEndDateTime	DATETIME	Yes	None	End before event
NewEndDateTime	DATETIME	Yes	None	End after event
PreviousStaffID	BIGINT UNSIGNED	Yes	FK -> Staff	Staff before event
NewStaffID	BIGINT UNSIGNED	Yes	FK -> Staff	Staff after event
PreviousServiceID	INT UNSIGNED	Yes	FK -> Service	Service before event
NewServiceID	INT UNSIGNED	Yes	FK -> Service	Service after event
Reason	VARCHAR(500)	Yes	Procedure validates when required	Cancellation, no-show, or correction explanation
RequestID	CHAR(36)	No	Indexed	UUID linking changes from one operation

Rules implemented Implements FR-10 through FR-12, FR-20, FR-21, FR-24, BR-13, BR-14, BR-21, and BR-24. Ordinary application roles receive INSERT through procedures only and no UPDATE or DELETE privilege.

Table PaymentMethod

Purpose Stores allowed non-sensitive payment method labels.

Column	Data type	Null allowed	Key or constraint	Description
PaymentMethodID	SMALLINT UNSIGNED	No	PK, AUTO_INCREMENT	Payment method identifier
MethodName	VARCHAR(40)	No	UNIQUE	Cash, Card Record, or Other
ActiveFlag	BOOLEAN	No	DEFAULT TRUE	Whether the method may be used for new transactions

Rules implemented Supports FR-13 and BR-17. No complete payment-card number or security code is stored.

Table PaymentTransaction

Purpose Stores non-sensitive charge and refund records associated with appointments.

Column	Data type	Null allowed	Key or constraint	Description
TransactionID	BIGINT UNSIGNED	No	PK, AUTO_INCREMENT	Payment transaction identifier
AppointmentID	BIGINT UNSIGNED	No	FK -> Appointment	Appointment receiving the transaction
Amount	DECIMAL(10,2)	No	CHECK > 0	Positive amount; type controls revenue direction
TransactionType	VARCHAR(10)	No	CHECK	Charge or Refund
PaymentMethodID	SMALLINT UNSIGNED	No	FK -> PaymentMethod	Allowed non-sensitive method
PaymentStatus	VARCHAR(12)	No	CHECK	Pending, Succeeded, Failed, or Voided
TransactionDate	DATETIME	No	NOT NULL	Business transaction date
RecordedByUserID	BIGINT UNSIGNED	No	FK -> UserAccount	Authorized staff or administrator
CreatedAt	DATETIME	No	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp
ExternalReference	VARCHAR(100)	Yes	UNIQUE	Optional unique receipt or external reference

Rules implemented Implements FR-13, FR-25, BR-17 through BR-19, BR-26, and BR-27. Only succeeded charges minus succeeded refunds contribute to revenue.

8 Normalization Analysis

Normal form	Design check	Evidence in this project
First Normal Form	Atomic columns and no repeating groups	Services qualified for a staff member are rows in StaffService. Appointment events and payments are separate rows rather than repeating columns.
Second Normal Form	Every non-key attribute depends on the whole key	The only composite-key tables are StaffService and AppointmentStatusTransition. Their flags describe the complete staff-service or from-status/to-status pair.
Third Normal Form	Non-key attributes do not depend on other non-key attributes	Role names, status names, payment methods, location details, account names, service details, and staff details are stored in their own tables and referenced by keys.

BookedPrice and BookedDurationMinutes are deliberate historical snapshots rather than accidental duplication. They describe the appointment as booked and prevent later Service updates from changing historical invoices, duration totals, and reports. AppointmentHistory similarly stores before-and-after values because its purpose is to preserve an immutable audit event.

Update anomaly avoided Staff names remain in UserAccount rather than being copied into Appointment. Changing a staff member's name therefore requires one update and does not rewrite appointment rows.

Delete anomaly avoided Services, staff, locations, and payment methods are deactivated rather than deleted when referenced by historical rows. Foreign keys use RESTRICT for protected parents.

9 Integrity Validation and Business Rule Enforcement

Rules that involve only one row use keys or CHECK constraints. Rules involving authorization, overlap, schedule precedence, history, or multiple tables use controlled procedures and transactions.

Rule	Requirement	Technical implementation	Verification
BR-01	Unique account ID and case-insensitive unique email	UserID PK; Email UNIQUE under utf8mb4_0900_ai_ci	Duplicate-email insert fails
BR-02	Exactly one valid primary role per account	RoleID NOT NULL FK to Role	Missing or invalid RoleID fails
BR-03	No plaintext passwords or sensitive card values	PasswordHash only; no card-number or security-code columns	Schema inspection and controlled-input test
BR-04	Nonnegative service price and positive duration	CHECK constraints	Insert invalid values and expect rejection
BR-05	Only active qualified staff may perform an active service	sp_BookAppointment joins active Staff, Service, and StaffService	Inactive or unqualified booking fails
BR-06	Future appointment with valid active references	FKs plus procedure validation	Past or inactive booking fails
BR-07	End equals start plus booked duration	CHECK using TIMESTAMPDIFF and procedure calculation	Mismatched end time fails
BR-08	No overlapping non-cancelled appointment for the same staff	Indexed overlap predicate inside locked transaction	Partial-overlap test fails
BR-09	Customer changes only own eligible appointment	Ownership, date, and status checks in procedures	Different-customer request fails
BR-10	Customer changes occur at least 24 hours before start	TIMESTAMPDIFF cutoff validation	Inside-cutoff request fails
BR-11	Appointment uses approved status	StatusID FK	Unknown status fails
BR-12	Status transitions follow matrix and create history	AppointmentStatusTransition plus sp_ChangeAppointmentStatus	Invalid transition fails with no history
BR-13	Controlled changes create immutable history	Transaction procedures insert AppointmentHistory	Compare appointment and history after change
BR-14	Cancelled and No-Show require reason	RequiresReason flag and procedure validation	Blank reason fails

Business Rules Continued

Rule	Requirement	Technical implementation	Verification
BR-15	Blocked time overrides date and recurring availability	Availability query precedence with NOT EXISTS filters	Blocked interval is absent
BR-16	Schedule has valid time range and permitted overlap	CHECK plus sp_SaveStaffSchedule	End not after start fails
BR-17	Valid positive payment and allowed values	CHECKs and PaymentMethod FK	Invalid amount, type, status, or method fails
BR-18	Revenue is succeeded charges minus succeeded refunds	vw_MonthlyRevenue conditional SUM	Pending, failed, and voided are excluded
BR-19	Only assigned staff or admin records payment	sp_RecordPayment role and assignment checks	Customer insert request fails
BR-20	Deactivation blocks new use but keeps history	Active checks; FKs use RESTRICT	New use fails and old records remain
BR-21	History is append-only	Privileges plus UPDATE and DELETE blocking triggers	Ordinary UPDATE and DELETE fail
BR-22	Times use seeded salon Eastern time	Location.TimeZone = America/New_York; application converts before write	Seed and time-zone query
BR-23	One service per appointment	ServiceID NOT NULL FK	Missing service fails
BR-24	Appointment operation and history are atomic	START TRANSACTION, handler ROLLBACK, COMMIT	Forced failure leaves no partial rows
BR-25	Customer phone and contact preference are required	Phone NOT NULL; preference CHECK	Missing phone or invalid preference fails
BR-26	Reschedule keeps AppointmentID and payment links	UPDATE existing appointment in sp_RescheduleAppointment	Payment remains on same AppointmentID
BR-27	Authorized payment allowed for any appointment status	sp_RecordPayment does not change appointment status	Cancelled and No-Show payment tests pass

Concurrency and Atomicity

MySQL does not provide a native exclusion constraint for time intervals. sp_BookAppointment and sp_RescheduleAppointment therefore run in a transaction, lock the relevant staff calendar rows with SELECT FOR UPDATE, evaluate the indexed overlap predicate, insert or update the appointment, append history, and then commit. An EXIT HANDLER executes ROLLBACK and re-signals the error if any validation or write fails.

Representative overlap predicate

```
SELECT AppointmentID FROM Appointment
WHERE StaffID = p_StaffID AND StatusID <> v_CancelledStatusID
  AND p_NewStart < EndDateTime AND p_NewEnd > StartDateTime
  AND AppointmentID <> COALESCE(p_AppointmentID, 0)
FOR UPDATE;
```

10 Database Objects and SQL Design

Object type	Object name	Purpose	Related requirements
View	vw_AppointmentDetails	Reusable joined appointment, customer, staff, service, location, and status details	FR-14, FR-16, FR-17
View	vw_DailySchedule	Daily schedule ordered by start time and filterable by staff and status	RQ-01
View	vw_CustomerAppointmentHistory	Customer appointment and payment summary newest first	RQ-03
View	vw_MonthlyRevenue	Succeeded charge totals minus succeeded refund totals by month	RQ-04, BR-18
View	vw_PopularServices	Appointment, completion, and payment totals by service	RQ-05
View	vw_StaffWorkload	Appointment counts, completions, and booked minutes by staff	RQ-06
View	vw_CancellationNoShow	Final outcome, reason, actor, and change timestamp	RQ-07
Procedure	sp_BookAppointment	Validates availability and overlap, inserts appointment and booking history atomically	FR-07 to FR-09, FR-18, FR-20
Procedure	sp_RescheduleAppointment	Checks ownership, cutoff, availability, and overlap; updates same ID and writes history	FR-10, BR-26
Procedure	sp_CancelAppointment	Checks actor, eligibility, cutoff, and reason; updates status and writes history	FR-10, FR-21
Procedure	sp_ChangeAppointmentStatus	Checks role, assignment, transition matrix, and required reason	FR-11, FR-12, FR-19
Procedure	sp_SaveStaffSchedule	Validates time range, schedule fields, and impermissible conflicts	FR-06, FR-22

Database Objects Continued

Object type	Object name	Purpose	Related requirements
Procedure	sp_RecordPayment	Checks role, assignment, values, method, and optional reference	FR-13, FR-25
Trigger	trg_AppointmentHistory_BlockUpdate	Rejects ordinary modification of audit rows	FR-24, BR-21
Trigger	trg_AppointmentHistory_BlockDelete	Rejects ordinary deletion of audit rows	FR-24, BR-21
Index	ux_UserAccount_Email	Case-insensitive email uniqueness and login lookup	FR-01, BR-01
Index	idx_Appointment_StaffStartStatus	Staff calendar and overlap validation	FR-07, FR-09, RQ-01
Index	idx_Appointment_CustomerStart	Ownership filter and newest-first customer history	FR-10, FR-14, RQ-03
Index	idx_Appointment_ServiceStartStatus	Service popularity and completion reporting	RQ-05
Index	idx_Payment_DateStatus	Revenue date-range and success-status filtering	RQ-04
Index	idx_Schedule_StaffDateTime	Date-specific and blocked-time availability lookup	FR-06, FR-07
Index	idx_Schedule_StaffDayTime	Recurring schedule lookup	FR-06, FR-07
Index	idx_History_AppointmentChanged	Chronological audit trail retrieval	FR-10 to FR-12, FR-24

Representative SQL Controls

Appointment timing and search index

```
CHECK (EndDateTime > StartDateTime),
CHECK (TIMESTAMPDIFF(MINUTE, StartDateTime, EndDateTime)
       = BookedDurationMinutes)

CREATE INDEX idx_Appointment_StaffStartStatus
ON Appointment (StaffID, StartDateTime, StatusID);
```

Append-only history triggers

```
CREATE TRIGGER trg_AppointmentHistory_BlockUpdate
BEFORE UPDATE ON AppointmentHistory
FOR EACH ROW
SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'AppointmentHistory is append-only';

CREATE TRIGGER trg_AppointmentHistory_BlockDelete
BEFORE DELETE ON AppointmentHistory
FOR EACH ROW
SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'AppointmentHistory is append-only';
```

Revenue calculation

```
SUM(CASE
      WHEN PaymentStatus = 'Succeeded'
       AND TransactionType = 'Charge' THEN Amount
      WHEN PaymentStatus = 'Succeeded'
       AND TransactionType = 'Refund' THEN -Amount
      ELSE 0
    END) AS NetRevenue
```

Implementation separation The final executable DDL, seed, procedure, view, trigger, test, and report statements should be submitted as SQL scripts. The TDD records the decisions and representative logic that those scripts must implement.

11 Query and Report Design

Query	Query or report	Main tables	Key SQL features	Expected result
RQ-01	Daily appointment schedule	Appointment, Customer, UserAccount, Staff, Service, AppointmentStatus	JOIN, date range, optional staff/status filters, ORDER BY StartDateTime	Start, customer, service, staff, duration, and status
RQ-02	Appointments by status	Appointment and detail tables	JOIN, WHERE status and date range	Matching appointment details
RQ-03	Customer appointment history	Appointment, Service, Staff, Status, PaymentTransaction	JOIN, LEFT JOIN aggregate, customer filter, newest first	Appointments with payment summary
RQ-04	Monthly revenue summary	PaymentTransaction	Conditional SUM, COUNT, AVG, year/month GROUP BY	Succeeded charges minus refunds; count and average charge
RQ-05	Popular services report	Service, Appointment, PaymentTransaction	LEFT JOIN, conditional aggregation, date filter	Appointment count, completed count, and revenue by service
RQ-06	Staff workload report	Staff, UserAccount, Appointment, Status	JOIN, COUNT, SUM duration, date and staff filters	Appointments, completions, and booked minutes by staff
RQ-07	Cancellation and no-show report	Appointment, History, Customer, Staff, Service, Status	JOIN, final-status filter, date range, actor lookup	Outcome, reason, actor, and event time

Representative daily schedule query

```
SELECT a.StartDateTime, cu.FirstName, cu.LastName,
       s.ServiceName, su.FirstName AS StaffFirstName,
       su.LastName AS StaffLastName,
       a.BookedDurationMinutes, ast.StatusName
FROM Appointment AS a
JOIN Customer AS c ON c.CustomerID = a.CustomerID
JOIN UserAccount AS cu ON cu.UserID = c.UserID
JOIN Staff AS st ON st.StaffID = a.StaffID
JOIN UserAccount AS su ON su.UserID = st.UserID
JOIN Service AS s ON s.ServiceID = a.ServiceID
JOIN AppointmentStatus AS ast ON ast.StatusID = a.StatusID
WHERE a.StartDateTime >= p_DayStart
  AND a.StartDateTime < p_DayEnd
ORDER BY a.StartDateTime;
```

12 Security Privacy and Backup Assumptions

- Development access is limited to the two project members through individual MySQL credentials.
- Application-style roles are simulated with least-privilege grants. Customers receive no direct table write privilege; staff receive only assigned-record operations through procedures; administrators receive approved maintenance procedures and reports.
- PasswordHash stores only a one-way salted hash string. The project does not store plaintext passwords, complete card numbers, card security codes, bank details, Social Security numbers, medical data, or real customer data.
- PaymentTransaction stores operational history only. It does not process funds or prove settlement with a payment gateway.
- AppointmentHistory update and delete are denied to ordinary roles and blocked by protective triggers.
- A production system would require TLS connections, secret management, audited role assignment, encrypted backups, scheduled full and incremental backups, restore testing, retention policies, monitoring, and incident procedures.
- For the class implementation, the team will export the schema, seed data, and test data with mysqldump before major demonstrations. Production-grade automation remains out of scope.

13 Test Plan and Acceptance Criteria

The team will execute each test against a clean seeded schema and capture the input statement, actual database result, and supporting query or screenshot. The Actual result column remains Pending execution until the SQL implementation is run.

Test	Requirement	Type	Test action	Expected result	Actual result
TC-01	FR-01 BR-01	Negative	Insert a second account with an existing email	Insert fails because email is unique	Pending execution
TC-02	FR-07 FR-08	Positive	Book active qualified staff for an open future time	Appointment and booking history are created with snapshots	Pending execution
TC-03	FR-09 BR-08	Negative	Attempt a partially overlapping booking for the same staff	Request fails and creates no rows	Pending execution
TC-04	FR-11 FR-12	Authorization	Customer attempts to mark an appointment No-Show	Status change is rejected	Pending execution
TC-05	FR-19	Workflow	Change Completed back to Scheduled	Transition fails and no history event is added	Pending execution
TC-06	FR-20 BR-24	Resiliency	Force history insertion or validation failure during booking	Full transaction rolls back	Pending execution
TC-07	FR-18	Historical	Change service price and duration after booking	Appointment snapshots remain unchanged	Pending execution
TC-08	FR-24 BR-21	Protection	Update or delete AppointmentHistory through ordinary role	Operation is rejected	Pending execution
TC-09	FR-10	Positive boundary	Reschedule owned Scheduled appointment more than 24 hours before start	Same appointment updates and prior values enter history	Pending execution
TC-10	FR-10 BR-10	Negative boundary	Customer reschedules less than 24 hours before start	Request fails and appointment remains unchanged	Pending execution

Test Plan Continued

Test	Requirement	Type	Test action	Expected result	Actual result
TC-11	FR-06 BR-15	Precedence	Add blocked time inside recurring or date availability	Blocked interval is not returned	Pending execution
TC-12	FR-13 FR-25	Positive	Assigned staff records a positive succeeded Charge	Payment row stores recorder and timestamp	Pending execution
TC-13	FR-25	Authorization	Customer attempts to create or edit payment	Operation is rejected	Pending execution
TC-14	BR-18 RQ-04	Report	Run revenue with charges, refund, failed, and pending rows	Only succeeded charge minus refund affects revenue	Pending execution
TC-15	FR-23	Deactivation	Deactivate a service with history and attempt a new booking	New booking fails and history remains	Pending execution
TC-16	FR-16	Query	Run daily schedule for one date and staff member	Only matching rows appear in start-time order	Pending execution
TC-17	FR-22 BR-16	Negative	Insert schedule with EndTime not later than StartTime	Record is rejected	Pending execution
TC-18	FR-03 BR-25	Negative	Insert Customer without phone or valid preference	Profile insert is rejected	Pending execution
TC-19	FR-10 BR-26	Relationship	Reschedule appointment that already has a payment	Same ID remains and payment stays linked	Pending execution
TC-20	FR-13 BR-27	Final status payment	Record refund for Cancelled and charge for No-Show	Both payment rows save without changing status	Pending execution

Acceptance Evidence

- DDL executes without error on MySQL 8.x.
- All seed rows, foreign keys, checks, unique constraints, views, procedures, triggers, and indexes are present.
- Every positive test produces the required rows and history.
- Every negative or authorization test fails without partial writes.
- The seven reports return correct filtered and aggregated results for the seeded test set.
- EXPLAIN output shows the planned indexes are available for the demonstrated query patterns.

14 Requirement Traceability

Functional requirement	Tables and relationships	Constraints or objects	Query report or test evidence
FR-01 to FR-03	Role, UserAccount, Customer	PKs, unique email, role FK, required phone, preference check	TC-01, TC-18; account and role queries
FR-04 to FR-06	Staff, Service, StaffService, StaffSchedule, Location	Active flags, composite key, schedule checks and precedence	TC-11, TC-17; qualification and availability results
FR-07 to FR-09	Appointment, StaffSchedule, StaffService, Service	sp_BookAppointment, overlap lock and indexes	TC-02, TC-03; open-time and booking evidence
FR-10	Appointment, AppointmentHistory, PaymentTransaction	sp_RescheduleAppointment, sp_CancelAppointment, retained AppointmentID	TC-09, TC-10, TC-19
FR-11 FR-12 FR-19 FR-21	AppointmentStatus, AppointmentStatusTransition, Appointment, History	Transition lookup, role and assignment checks, required reason	TC-04, TC-05; before-and-after status queries
FR-13 FR-25	PaymentTransaction, PaymentMethod, Appointment	FKs, checks, sp_RecordPayment	TC-12, TC-13, TC-20
FR-14	UserAccount, Appointment, PaymentTransaction	Ownership and assignment filters in views or application queries	Customer, staff, and administrator result comparison
FR-15	All operational tables	Administrative procedures and least-privilege grants	Administrative maintenance demonstrations

Functional requirement	Tables and relationships	Constraints or objects	Query report or test evidence
FR-16 FR-17	Appointment, PaymentTransaction, Service, Staff, History	Seven views or report queries and supporting indexes	TC-14, TC-16; report output and EXPLAIN
FR-18	Appointment, Service	BookedPrice and BookedDurationMinutes snapshots	TC-07
FR-20	Appointment, AppointmentHistory	Atomic stored procedures with rollback handlers	TC-06
FR-22	StaffSchedule	CHECK constraints and sp_SaveStaffSchedule	TC-17
FR-23	Service, Staff, StaffService, Location, Appointment	Active checks with RESTRICT history-preserving FKs	TC-15
FR-24	AppointmentHistory	Restricted privileges and blocking triggers	TC-08

Implementation Checklist

- Create the hair_fix_salon schema and all 14 tables in dependency order.
- Insert Role, Location, AppointmentStatus, AppointmentStatusTransition, and PaymentMethod seed rows.
- Create constraints and indexes, then verify their metadata.
- Create procedures, views, and append-only history triggers.
- Load fictional customers, staff, services, qualifications, schedules, appointments, history, and payments.
- Execute TC-01 through TC-20 and record actual results.
- Run RQ-01 through RQ-07 and capture output and EXPLAIN plans.
- Update the TDD Actual result column and submit the final PDF with the SQL scripts and ERD required by the instructor.